

MicroVMs auf Proxmox VE

Minimal-invasiver QEMU-MicroVM-Patch, Benchmarks & Praxiserfahrungen

FrOSCon 2026 · Hochschule Bonn-Rhein-Sieg · Alexander Wirt

Wer bin ich?

Alexander Wirt – CTO und Open-Source-Entwickler

- Debian Developer seit 2002
- Arbeit bei **credativ** – Open-Source-Beratung und Linux-Infrastruktur
- Schwerpunkte: Virtualisierung, Hochverfügbarkeit, Proxmox VE

Wie es dazu kam: Mein Kollege Florian meinte beiläufig: „MicroVMs in Proxmox — das müsste man mal machen.“ Ich bin prompt darauf angesprungen. Dieser Vortrag ist, was daraus wurde.

Agenda

- **Motivation:** Die Lücke zwischen Container (LXC) und voller VM
- **Isolationsspektrum & Kandidaten:** QEMU (q35), QEMU microVM, Firecracker
- **Wie Proxmox VMs startet:** `pve-qemu-server`, `config_to_command` & PCI-Topologie
- **Technische Hürden in PVE:** PCI-Topologie, SeaBIOS & Legacy-Geräte
- **Der Patch:** Minimal-invasive Integration in Perl-Backend & ExtJS-GUI
- **Test-Pipeline:** `.deb` -Packaging & Nested-PVE-Testbed
- **Messungen & Benchmarks:** Startup-Zeiten, VMM-RSS, Dichte & Kernel-Einfluss
- **Live-Demo:** Erstellung im Web-UI & Konsole
- **Einordnung, Grenzen & Fazit:** Wann nimmt man was?

Motivation: Container vs. VM

Container (LXC / Docker)

- Starten in Millisekunden
- Fast kein Speicher-Overhead
- **Einschränkung:** Geteilter Host-Kernel. Ein Kernel-Exploit kompromittiert direkt den Hypervisor-Host.

KVM-VMs (Standard PC / Q35)

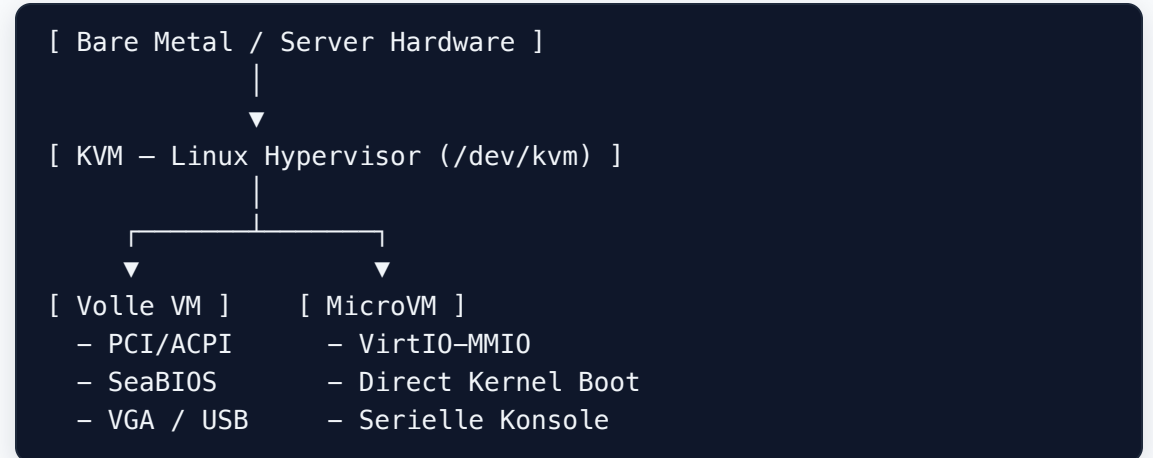
- Harte Hardware-Isolation & eigener Kernel
- Eigene Sicherheitsgrenze
- **Einschränkung:** Träge Bootzeiten (5–30 s), höherer Memory-Footprint (~140 MB RSS).

Der Anwendungsfall: Kurzlebige Workloads (CI/CD Runner, FaaS, Multi-Tenant Build-Umgebungen). Container bieten zu wenig Isolation, Standard-VMs sind zu schwergewichtig.

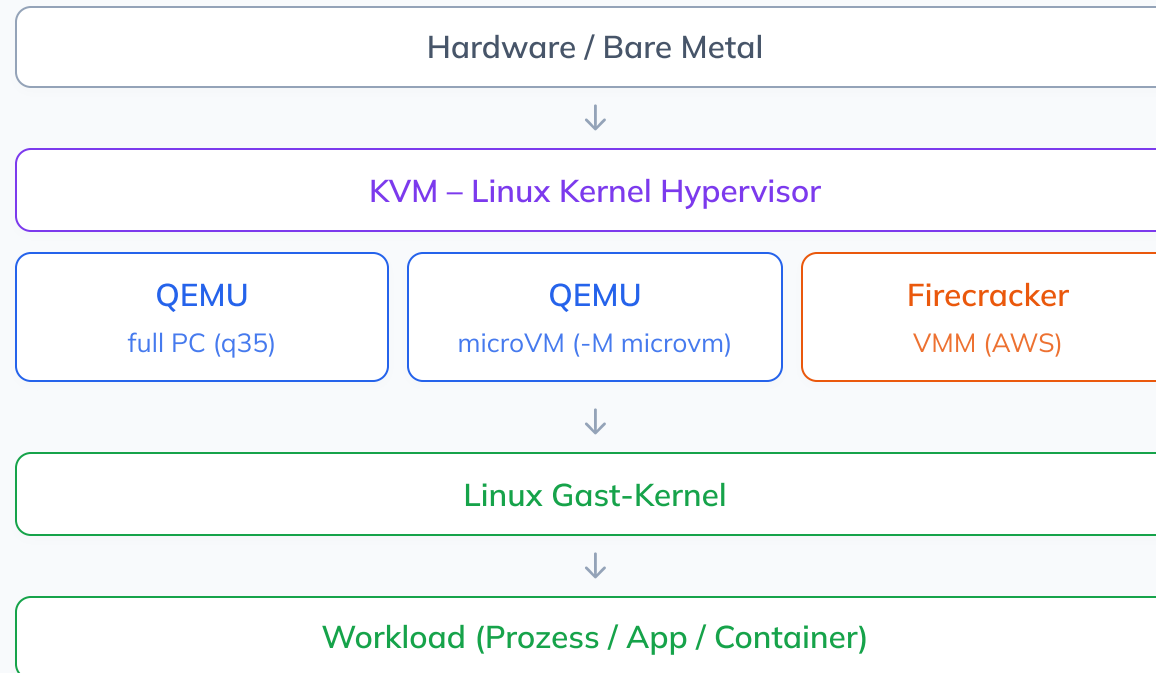
Was eine MicroVM ausmacht

Eigenschaften von MicroVMs:

- **VM-Isolation:** Eigener Linux-Kernel via KVM (harte Sicherheitsgrenze)
- **Schneller Start:** Bootzeiten von ~150–200 ms
- **Reduzierte Angriffsfläche:** Keine Emulation von Floppy, IDE, VGA, PCI oder USB – historische Hauptquellen von QEMU-Sicherheitslücken (CVEs)
- **Schlanker Footprint:** Nur minimale VirtIO-MMIO-Geräte am Systembus



Isolationsspektrum: Ein KVM, drei VMMs



KVM ist bei allen dreien identisch. Sie unterscheiden sich primär im VMM (Device-Modell, Firmware, Angriffsfläche).

Die drei VMM-Kandidaten

Eigenschaft	QEMU full (q35)	QEMU microVM (-M microvm)	Firecracker
Sprache / LOC	C · ~2.000.000 Zeilen	C · ~2.000.000 Zeilen	Rust · ~50.000 Zeilen
Device-Transport	VirtIO-PCI & emuliert	VirtIO-MMIO	VirtIO-MMIO
Firmware / BIOS	SeaBIOS / OVMF	qboot / Direct Boot	Keins (Direct Boot)
Emulierte Geräte	Dutzende (PCI, USB, IDE, VGA)	Minimal (nur VirtIO)	Exakt 5 Geräte
Proxmox-Integration	Standard (nativ)	Über Patch machbar	Neuer VMM / Fremdkörper

Der Punkt für Proxmox: QEMU microVM steckt bereits im `qemu-system-x86_64`, das Proxmox ohnehin mitliefert. Es braucht keinen neuen VMM, nur die Freischaltung in Proxmox.

Firecracker: Das radikale VMM-Modell

Firecracker-Design (AWS):

- **Strikter Minimalismus:** Nur 5 MMIO-Geräte, kein ACPI, kein PCI, kein BIOS.
- **REST-API-Steuerung:** Eigener Prozess pro VM, Konfiguration über HTTP/Socket.
- **Ziel:** Maximale Dichte & Sub-Sekunden-Starts für AWS Lambda und Fargate.

Die Kehrseite in der Praxis:

- **Kein Storage-Layer:** Nur rohe Disk-Dateien (kein QCOW2, LVM, ZFS-Thin).
- **Keine Standard-Tools:** Kein QMP, kein Guest-Agent, kein Monitoring.
- **Hoher Integrationsaufwand:** Sämtliche Infrastruktur (Netzwerk, Storage, Lifecycle) muss drumherum selbst gebaut werden.

Praxiserfahrung: Mehrere Plattform-Teams, die mit Firecracker gestartet sind, wechselten für flexiblere Workloads zu QEMU/MicroVMs zurück – weil Firecrackers bewusste Reduktion außerhalb von FaaS schnell zur harten Grenze wird.

Wie Proxmox eine VM startet

```

/etc/pve/qemu-server/<vmid>.conf
    |
    ▼
[ PVE::QemuServer::config_to_command() ]
    |
    ├── PVE::QemuServer::Machine (Maschinentyp & Versionierung)
    ├── PVE::QemuServer::PCI      (PCIe Root Ports & Bridges)
    ├── PVE::QemuServer::Drive    (Laufwerke: SCSI, VirtIO-PCI)
    └── PVE::QemuServer::Memory    (RAM & Hugepages)
    |
    ▼
qemu-system-x86_64 -id 100 -name vm100 ...

```

Proxmox übersetzt die deklarative Konfigurationsdatei via Perl in den exakten QEMU-Kommandozeilenaufruf.

Was Proxmox standardmäßig generiert: `qm showcmd`

```
/usr/bin/kvm -id 100 -name vm100 -chardev socket,id=qmp,path=/var/run/qemu-server/100.qmp ... \  
-nodefaults -boot menu=on,strict=on,splash=/usr/share/qemu-server/bootsplash.jpg \  
-vga none -nographic \  
-device pcie-root-port,id=pci.1,bus=pcie.0,slot=1 ... \  
-device pcie-root-port,id=pci.2,bus=pcie.0,slot=2 ... \  
-device virtio-balloon-pci,id=balloon0,bus=pci.1,addr=0x0 \  
-device virtio-scsi-pci,id=scsihw0,bus=pci.2,addr=0x0 \  
-drive file=/var/lib/vz/images/100/vm-100-disk-0.raw,if=none,id=drive-scsi0 ...
```

Struktur des Aufrufs:

1. Proxmox initialisiert standardmäßig eine vollständige **PCIe-Root-Port- und Bridge-Hierarchie**.
2. Alle Geräte (`scsi`, `net`, `balloon`) werden als **PCI-Geräte** (`-pci`) an Slots gehängt.
3. Als Boot-Mechanismus wird das Standard-BIOS (SeaBIOS/OVMF) vorausgesetzt.

Warum microvm in PVE nicht out-of-the-box läuft

Was Proxmox generiert:

- PCIe Root Ports (`pcie.0` , `pci.1` , `pci.2`)
- PCI-basierte Geräte (`virtio-blk-pci` , `virtio-net-pci`)
- SeaBIOS / OVMF Bootloader
- ACPI-Power-States (`PIIX4_PM` , `ICH9-LPC`)
- Standard-Grafik & USB-Tablets

Was `microvm` erfordert:

- Kein PCI-Bus (Geräte via `virtio-mmio`)
- Kein SeaBIOS (Direct Kernel Boot)
- Keine ACPI-Legacy-Geräte
- Reine serielle Konsole (`ttyS0` / `hvc0`)
- Minimaler Device-Tree

Problem im Standard-PVE: Setzt man manuell `machine: microvm` , schlägt die Schema-Prüfung fehl, Proxmox baut PCI-Bridges für einen nicht existierenden PCI-Bus und QEMU bricht den Start mit Fehlermeldung ab.

Der Patch: Architektur & Designentscheidungen

Voraussetzung: Null Risiko für bestehende `pc` / `q35` VMs. Alle Änderungen greifen nur, wenn explizit `machine: microvm` konfiguriert ist.

BACKEND (`PVE-QEMU-SERVER`)

- `Machine.pm`: Erlaubt `microvm` in der JSON-Schema-Validierung.
- `Machine.pm`: Führt `microvm` als eigenständigen BaseType.
- `config_to_command`: Überspringt PCI-Bridge-Generierung und ACPI-Power-Flags bei MicroVMs.

WEB-GUI (`PVE-MANAGER`)

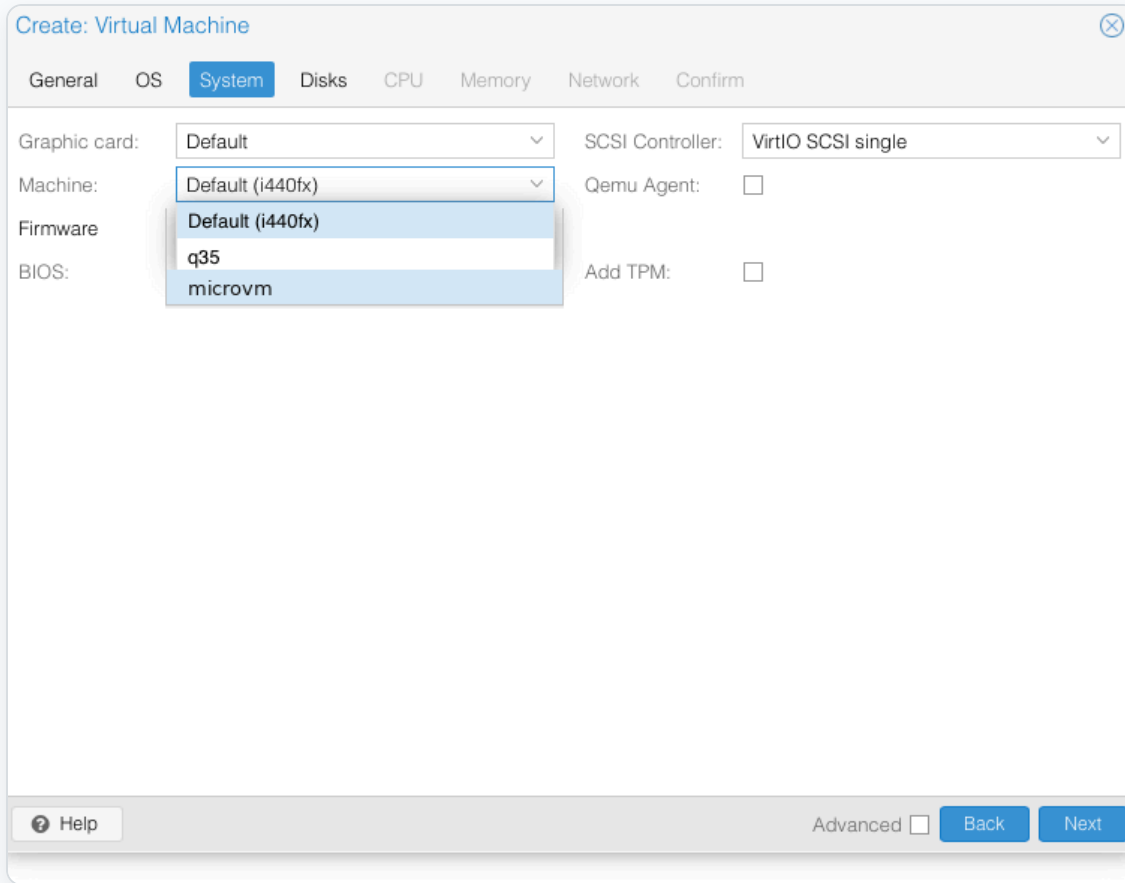
- `QemuMachineSelector.js`: Fügt `microvm` zur Auswahlliste hinzu.
- `Architecture.js`: Erlaubt `microvm` in der x86_64-Architektur-Filterliste.
- `MachineEdit.js`: Verhindert Zurücksetzen auf Default.

Der Backend-Patch im Detail: PVE::QemuServer::Machine

```
--- a/src/PVE/QemuServer/Machine.pm
+++ b/src/PVE/QemuServer/Machine.pm
@@ -58,7 +58,7 @@ my $machine_fmt = {
    description => "Specifies the QEMU machine type.",
    type => 'string',
    pattern =>
-       '(pc|pc(-i440fx)?-\\d+(\\.\\d+)(\\+pve\\d+)?(\\.pxe)?|q35|pc-q35-\\d+(\\.\\d+)(\\+pve\\d+)?(\\.pxe)?|virt(?:-\\d+(\\.\\d+)+)?(\\+pve\\d+)? )',
+       '(pc|pc(-i440fx)?-\\d+(\\.\\d+)(\\+pve\\d+)?(\\.pxe)?|q35|pc-q35-\\d+(\\.\\d+)(\\+pve\\d+)?(\\.pxe)?|virt(?:-\\d+(\\.\\d+)+)?(\\+pve\\d+)?|microvm) ',
    maxLength => 40,
@@ -167,6 +167,7 @@ my sub machine_base_type {
    return 'q35' if $machine_type =~ m/q35/;
    return 'i440fx' if $machine_type =~ m/^pc/;
    return 'virt' if $machine_type =~ m/^virt/;
+   return 'microvm' if $machine_type =~ m/^microvm/;
```

Ergebnis: Proxmox akzeptiert `machine: microvm` in CLI, GUI und REST-API und hängt automatisch `+pve0` zur Revisionskontrolle an.

MicroVM in der Proxmox Web-GUI



Create: Virtual Machine

General OS **System** Disks CPU Memory Network Confirm

Graphic card: Default

Machine: Default (i440fx)

Firmware: Default (i440fx)

BIOS: q35

SCSI Controller: VirtIO SCSI single

Qemu Agent: ☐

Add TPM: ☐

microvm

Help Advanced ☐ Back Next

Auswirkungen in der Web-GUI:

- **VM-Erstellungs-Wizard:** Machine-Dropdown bietet neben *Default (i440fx)* und *q35* nun **microvm** an.
- **VM-Hardware-Optionen:** Machine-Typ bleibt nach dem Speichern dauerhaft in `/etc/pve/qemu-server/<vmid>.conf` erhalten.
- **ExtJS-Kategorieprüfung:** `allowedMachines.x86_64` validiert die Auswahl fehlerfrei.

Test-Pipeline: `.deb` -Build & Nested-PVE

Sicheres Testen von PVE-Patches ohne Eingriff in Produktivknoten:

```
[ Git Repo: Patch-Dateien ]
    |
    ▼ (Ansible Orchestrator)
[ 1. Build-VM: Debian Trixie / Bookworm ]
├─ quilt push (Patches anwenden)
├─ dpkg-buildpackage -b -uc -us → *.deb
    |
    ▼ (Scp / Fetch Artefakte)
[ 2. Nested Proxmox-Test-VM (VM 9002) ]
├─ dpkg -i *.deb
├─ systemctl restart pvedaemon pveproxy
└─ qm showcmd & Boot-Verifikation
```

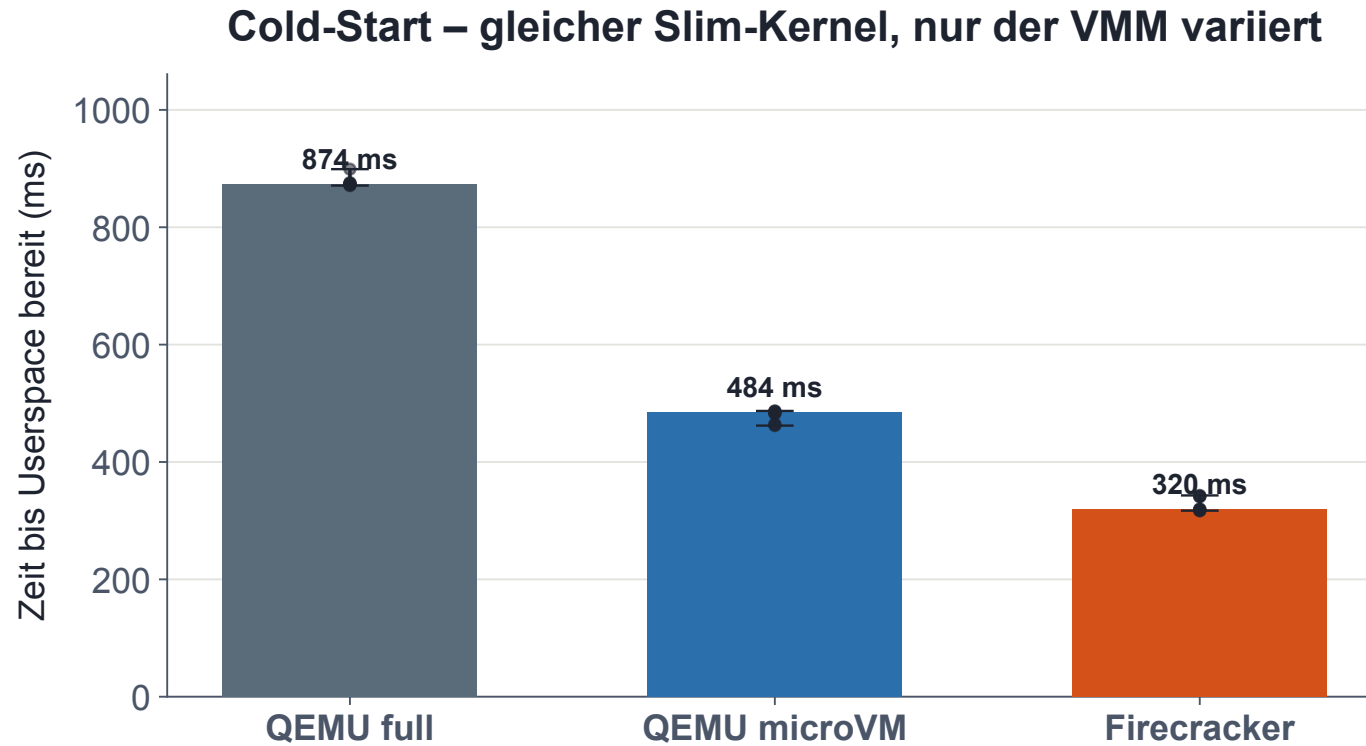
Nested-KVM-Detail: Die äußere Test-VM muss mit `--cpu host` angelegt werden, damit Hardware-Virtualisierung (`/dev/kvm`) in der inneren Test-VM ankommt.

Benchmarks: Startup-Zeiten & Ressourcenbedarf

Messaufbau & Methodik (reiner VMM-Vergleich):

- **Identischer Gast-Kernel:** selbst gebauter Slim-Kernel (~7 MB bzImage, ACPI=y, keine Module).
- **Identisches RootFS:** Alpine Linux 3.21 minrootfs (ext4).
- **Readiness-Signal:** Zeit bis zum seriellen Marker `+++BENCHREADY+++` aus dem Gast-Init.
- **Hinweis:** Die absoluten Zeiten variieren je nach Host-Hardware — relevant ist das relative Verhältnis der Architekturen zueinander.

Cold-Start: Gleicher Kernel, nur VMM variiert

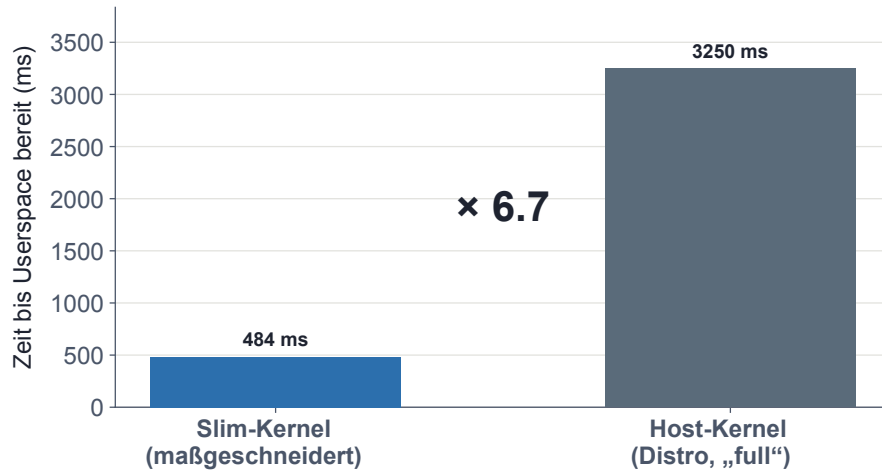


AMD Ryzen 5 5600H with Radeon Graphics · Kernel Slim Linux version 6.1.102 (identisch für alle) · Full-Vergleich 7.0.0-3-pve · 2026-06-15

Mit demselben schlanken Kernel schmilzt der oft zitierte „Faktor 100“ zusammen: QEMU microvm startet in rund **180 ms**.

Der Kernel-Einfluss: Slim-Kernel vs. Distro-Kernel

Kernel-Vergleich – dieselbe QEMU microVM (Slim- vs. Host-Kernel)



AMD Ryzen 5 5600H with Radeon Graphics · Kernel Slim Linux version 6.1.102 (identisch für alle) · Full-Vergleich 7.0.0-3-pve · 2026-06-15

Dieselbe MicroVM – nur Kernel getauscht:

- Schlanker Kernel (Slim): ~180 ms
- Standard-Distro-Kernel (Fat): ~1.300 ms

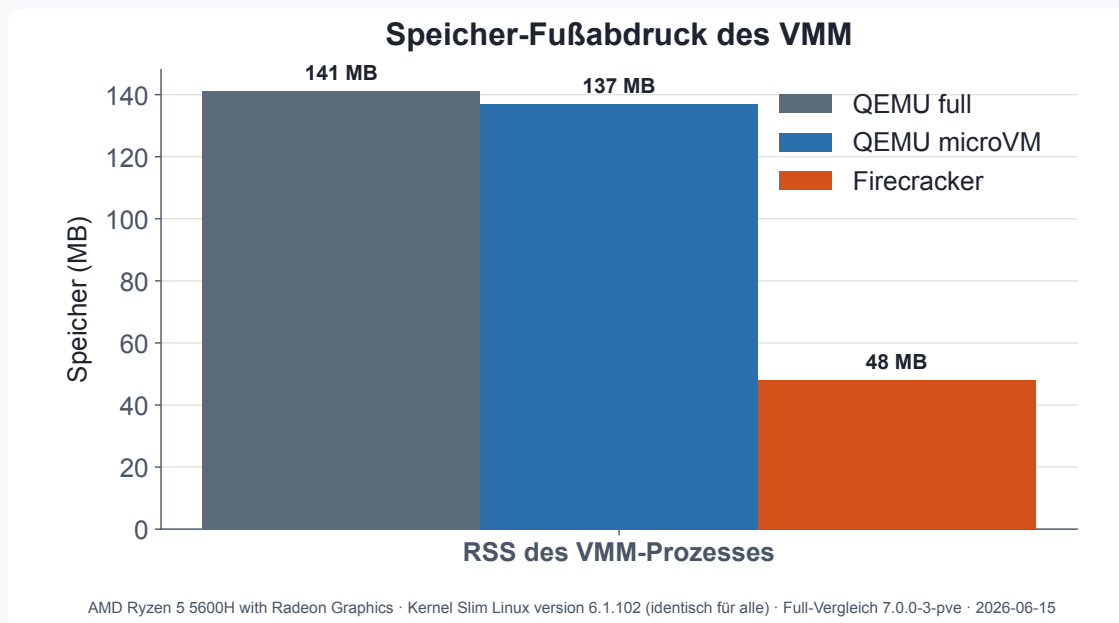
Erkenntnis:

Der VMM-Wechsel (q35 → microvm) spart ~1.000 ms.

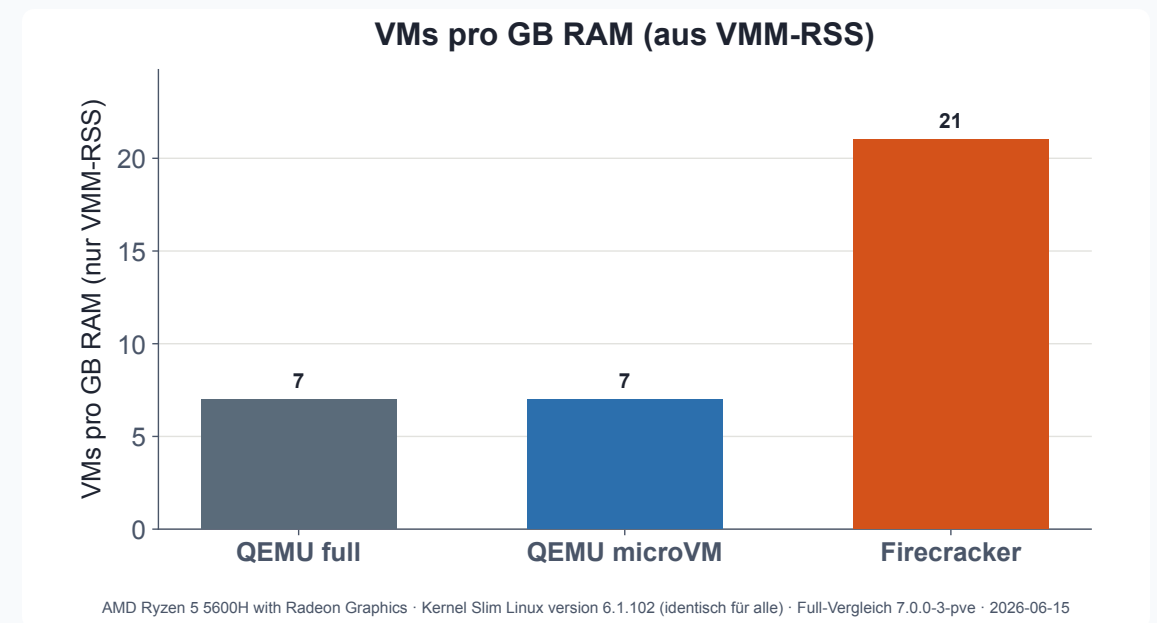
Der **Kernel-Wechsel** spart weitere ~1.100 ms.

Das größte Optimierungspotenzial liegt im Gast-Kernel.

Memory Footprint (VMM RSS) & Dichte

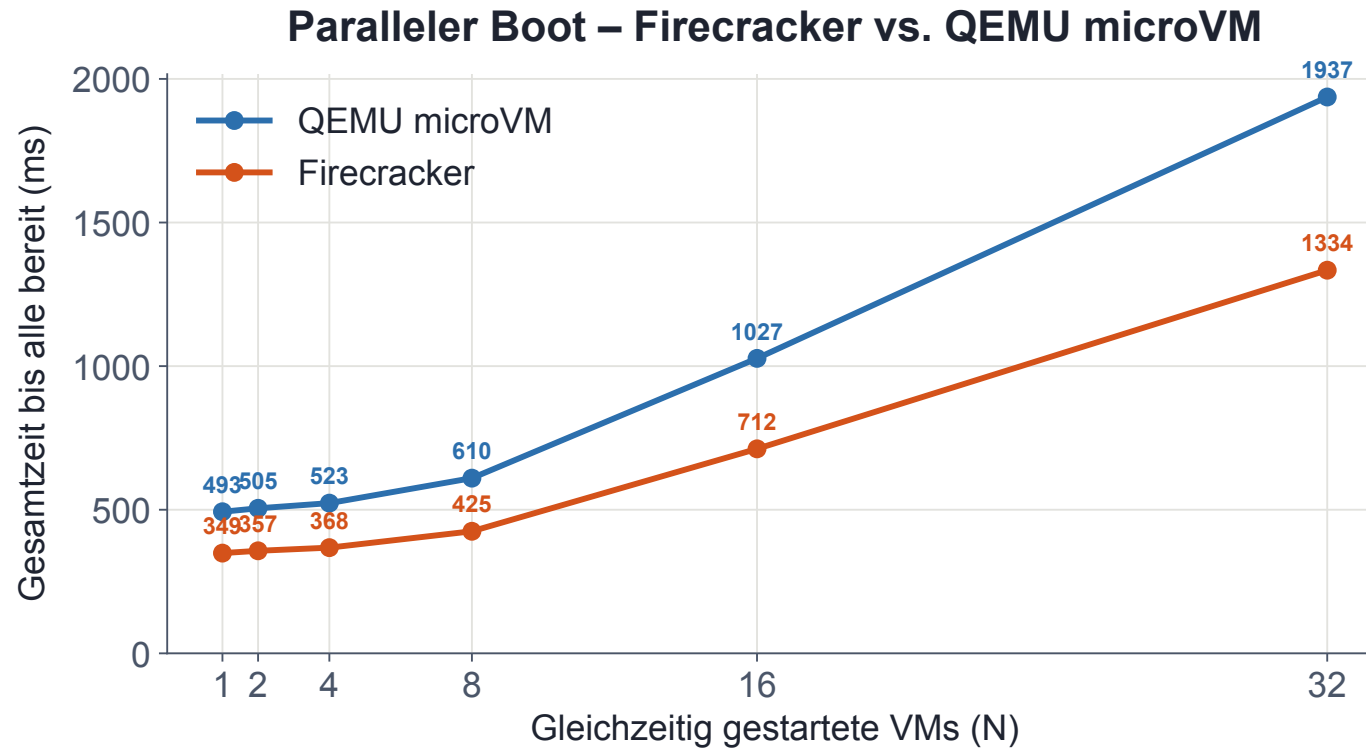


- **QEMU Full (q35):** ~141 MB RSS
- **QEMU MicroVM:** ~137 MB RSS (*nur ~4 MB weniger*)
- **Firecracker:** ~48 MB RSS



- **QEMU full / microvm:** ~7 VMs / GB RAM
- **Firecracker:** ~21 VMs / GB RAM (~3× mehr)
- **Grund:** QEMUs C-Codebasis belegt Grundspeicher; Dichte bleibt Firecrackers Domäne.

Skalierung: Paralleler Start (1 bis 32 VMs)



AMD Ryzen 5 5600H with Radeon Graphics · Kernel Slim Linux version 6.1.102 (identisch für alle) · Full-Vergleich 7.0.0-3-pve · 2026-06-15

QEMU microvm skaliert beim parallelen Start bis 32 gleichzeitige VMs nahe an Firecracker.

Live-Demo: MicroVM in Proxmox VE

Ablauf der Demo:

1. Proxmox Web-UI aufrufen (`https://localhost:8006`)
2. **Create VM Wizard** -> Tab *System* -> Machine:
`microvm`
3. Direct Kernel Boot konfigurieren (`vmlinuz-slim` + `rootfs`)
4. Starten & Boot-Log in `xterm.js` / Serial Console betrachten

VM-Konfiguration (`/etc/pve/qemu-server/100.conf`):

```
name: microvm-demo
machine: microvm
cores: 2
memory: 512
kernel: /var/lib/vz/template/qemu/vmlinuz-slim
args: console=ttyS0 root=/dev/vda rw
virtio0: local-lvm:vm-100-disk-0,size=4G
net0: virtio=BC:24:11:AA:BB:CC,bridge=vbr0
serial0: socket
vga: serial0
```

QEMU-Aufruf: `qm showcmd` im Detail

```
# qm showcmd 101 (Gepatchtes Proxmox VE)
/usr/bin/kvm -id 101 -name microvm-demo -daemonize -nodefaults -nographic \
-machine 'smm=off,type=microvm+pve0' \
-cpu host -smp 2,sockets=1,cores=2 -m 512 \
-kernel /var/lib/vz/template/qemu/vmlinuz-slim \
-append 'console=ttyS0 root=/dev/vda rw init=/init quiet' \
-chardev socket,id=serial0,path=/var/run/qemu-server/101.serial0,server=on,wait=off \
-device 'isa-serial,chardev=serial0' \
-blockdev '{"driver":"raw","file":{"aio":"io_uring","filename":".../vm-101-disk-0.raw"},"node-name":"drive-virtio0"}' \
-device 'virtio-blk-device,drive=drive-virtio0,id=virtio0' \
-netdev 'type=tap,id=net0,ifname=tap101i0,script=/var/lib/qemu-server/pve-bridge,...' \
-device 'virtio-net-device,mac=bc:24:11:aa:bb:cc,netdev=net0,id=net0' \
-device 'virtio-balloon-device,id=balloon0,free-page-reporting=on'
```

- **microvm+pve0** : Kein PCI-Root, kein ACPI S3/S4
- **virtio-*-device** : VirtIO-MMIO für Disk, Net & Balloon
- **isa-serial** : Serielle Konsole über UNIX-Socket
- **Keine PCI-Bridges: pci.0** & Bridges übersprungen
- **Kein USB/VGA**: Keine Controller initialisiert
- **Direct Kernel Boot**: Überspringt SeaBIOS/OVMF

QEMU Device Tree

```
main-system-bus (MicroVM)
├── isabus-bridge
│   └── isa.0
│       └── isa-serial (ttyS0)
├── virtio-mmio (24 statische MMIO-Slots)
│   ├── [Slot 21] — virtio-net-device ("net0" via TAP)
│   ├── [Slot 22] — virtio-blk-device ("virtio0")
│   └── [Slot 23] — virtio-balloon-device ("balloon0")
├── acpi-ged (ACPI Generic Event Device)
└── ioapic (x2), kvmclock, fw_cfg_io
```

- **Kein PCI-Root-Bus (`pci.0`):** Null PCI-Initialisierungszyklen beim Start.
- **VirtIO-MMIO Transport:** Feste MMIO-Adressen am Systembus statt dynamischer PCI-BAR-Zuweisung.
- **Minimaler Footprint:** Keine USB-, IDE-, SATA-, VGA- oder Floppy-Controller im Baum.

Einordnung: LXC, MicroVM, volle VM

Wann nimmt man in der Praxis was?

Kriterium	LXC Container	QEMU microVM (PVE)	Volle KVM-VM (q35)
Isolation	Geteilter Kernel (weich)	Eigener Kernel (hart)	Eigener Kernel (hart)
Bootzeit	< 100 ms	~180 ms	5 – 30 s
VMM Memory RSS	0 MB	~137 MB	~141 MB
Live-Migration	Nein	Nein / eingeschränkt	Ja (vollständig)
Hardware-Passthrough	Eingeschränkt	Nein (kein PCI)	Ja (PCIe, GPU, USB)
Typischer Use-Case	Webserver, Linux-Dienste	CI/CD Runner, FaaS, Sandboxes	Windows, Datenbanken, Cluster

Grenzen & Einschränkungen

- **Keine Live-Migration:** MicroVMs sind in QEMU nicht migrations-versioniert. Für Cluster-Updates ungeeignet.
- **Kein Hotplug:** Geräte, vCPUs und RAM können nicht im laufenden Betrieb hinzugefügt werden.
- **Kein Windows / Non-Linux:** MicroVMs erfordern Linux-Kernel mit VirtIO-MMIO-Treibern.
- **Kein PCI-Passthrough:** Für dedizierte GPUs oder PCIe-NICs bleibt nur `q35`.

Stärke bei kurzlebigen Workloads: MicroVMs passen zu Jobs, die nach Sekunden oder Minuten wieder verworfen werden (CI/CD, Tests, Microservices).

Was noch fehlt: Offene Punkte & Leftovers

- **Direct Kernel Boot in `qm.conf`**: Parameter wie `kernel`, `initrd` und `append` werden bisher über `args:` durchgereicht. Es fehlen native First-Class-Config-Optionen in CLI, REST-API und GUI.
- **Storage- & Image-Handling**: Wie kommen schlanke Kernel und RootFS-Templates auf den Node? (Echtes MicroVM-Image-Format oder Storage-Plugin-Erweiterung).
- **Netzwerk via MMIO**: `virtio-net-device` sauber an Linux-Bridges/VLANs anbinden und im Erstellungs-Wizard konfigurierbar machen.
- **GUI-Bereinigung**: Nicht unterstützte Optionen (`scsihw`, `bios`, `vga`, `ide/sata`) bei `machine: microvm` in der Web-UI ausblenden statt mit unpassenden Defaults anzuzeigen.
- **Upstream-Diskussion**: Abstimmung mit dem Proxmox-Entwicklerteam auf `pve-devel` über die saubere Modellierung im PVE-Objektmodell.

Zusammenfassung & Fazit

1. **Isolation & Performance:** MicroVMs verbinden KVM-Sicherheit mit Startzeiten von unter 200 ms.
2. **Der Kernel ist der Hebel:** Ein 60-MB-Distro-Kernel braucht Sekunden für Udev & Hardware-Scan. Erst ein schlanker All-Built-in-Kernel (<10 MB) ohne Initramfs bringt das Tempo.
3. **VirtIO-MMIO als Paradigma:** Verzicht auf PCI spart Initialisierungszeit und Speicher, limitiert aber auf statische, nicht-hotplugfähige Geräte.
4. **Machbarkeit bewiesen:** Ein minimal-invasiver Patch ohne Seiteneffekte für bestehende VMs beweist, dass Proxmox VE MicroVMs nativ unterstützen kann.

Vielen Dank! Fragen & Diskussion

Alexander Wirt · `alexander.wirt@credativ.de`

Code & Patches: <https://github.com/credativ/FrOSCon2026-microvm>

FrOSCon 2026 · Hochschule Bonn-Rhein-Sieg

